Quantum-Assisted Physics-Informed Neural Networks for Burgers' Equation

BQP x WISER Quantum Challenge 2026 -- Technical Report

Team: Dennis Appiah Kubi (Role 1 : Theory), Ajay Sankar Makkena (Role 2 : Implementation), Pascal Chabo Bya'ombe (Role 3 : - XAI and Documentation)

https://github.com/mas622424/WISER-BQP-QAPINN

1. Introduction

Partial differential equations (PDEs) are widely used to model physical phenomena such as fluid flow, heat transfer, and wave propagation. Traditional numerical methods, including finite difference and finite element methods, have been successfully applied to solve these equations but can become computationally expensive for nonlinear problems. Physics-Informed Neural Networks (PINNs) have emerged as an alternative approach by embedding the governing physical laws directly into the neural network training process, enabling the network to approximate solutions while satisfying the underlying PDE.

To further enhance the learning capability of PINNs, Variational Quantum Circuits (VQCs) can be integrated into the framework, resulting in a Quantum-Assisted Physics-Informed Neural Network (QAPINN). This report presents the one-dimensional viscous Burgers' equation as the benchmark problem, derives it from the Navier-Stokes equation, formulates the PINN and QAPINN frameworks mathematically, and evaluates both approaches through a detailed Explainable AI (XAI) analysis. Rather than aiming to prove that QAPINNs categorically outperform classical PINNs, this study focuses on explaining when, why, and how the introduction of a quantum layer changes the learning dynamics of a PINN, comparing a classical baseline (cPINN) against QAPINN configurations at 3, 4, and 5 qubits.

2. Physics Formulation

The physical model used throughout this study is derived from the principle of conservation of momentum. According to Newton's second law, the rate of change of momentum of a fluid element equals the total forces acting on it.

2.1 Conservation Principle and Governing Equation

For one-dimensional fluid flow, the velocity field is u = u(x,t), where x is the spatial coordinate and t is time. The total change in velocity experienced by a moving fluid particle is given by the material derivative, Du/Dt = du/dt + u(du/dx), combining local acceleration (du/dt) and convective acceleration (u du/dx). Applying Newton's second law to a fluid element (F = ma) and substituting the fluid density (rho = m/V) gives the force per unit volume as F = rho (Du/Dt).

Two forces are considered: the pressure force and the viscous force. The net pressure force per unit mass is derived via a Taylor expansion of the pressure field p(x,t) across a fluid element, giving Fp/m = -(1/rho)(dp/dx). The viscous force is derived from Newton's law of viscosity (shear stress tau = mu du/dx) and a similar Taylor expansion of the shear stress across the element, giving Fv/m = nu (d2u/dx2), where nu = mu/rho is the kinematic viscosity.

Summing all forces per Newton's second law yields the one-dimensional Navier-Stokes equation: du/dt + u du/dx = -(1/rho) dp/dx + nu d2u/dx2. Assuming a negligible pressure gradient (dp/dx = 0), this reduces to the one-dimensional viscous Burgers' equation:

du/dt + u du/dx - nu d2u/dx2 = 0

The nonlinear convection term (u du/dx) causes the solution to steepen over time, allowing faster-moving fluid to catch up with slower-moving fluid and produce sharp gradients or shock-like structures. The viscous diffusion term (nu d2u/dx2) acts in the opposite direction, smoothing steep gradients by transferring momentum between neighboring fluid layers. The competition between these two terms is what produces Burgers' equation's characteristic sharp shock, which is central to the XAI analysis in Section 8.

2.2 Initial and Boundary Conditions

The computational domain is x in [a,b], t in [0,T], with initial condition u(x,0) = u0(x) and boundary conditions u(a,t) = g1(t), u(b,t) = g2(t). For the benchmark problem used in this study, the domain is x in [-1,1], t in [0,1], with initial condition u(x,0) = -sin(pi x) and homogeneous Dirichlet boundary conditions u(-1,t) = 0, u(1,t) = 0.

2.3 Why Burgers' Equation Is Chosen

The one-dimensional viscous Burgers' equation is selected as the benchmark because it captures the essential physical phenomena of nonlinear convection and viscous diffusion while remaining mathematically tractable. It is one of the most widely used benchmark equations for validating numerical methods and PINNs due to its known analytical solution under standard initial and boundary conditions. Its nonlinear nature also makes it an appropriate test problem for evaluating the QAPINN framework.

3. Physics-Informed Neural Network (PINN) Formulation

PINNs are a class of neural networks that solve PDEs by incorporating the governing physical laws directly into the training process, rather than relying solely on labeled data. A PINN approximates the unknown exact solution u(x,t) with a neural network u_theta(x,t), where theta represents all trainable parameters (weights W and biases b).

3.1 Neural Network Structure

The network maps inputs (x,t) through hidden layers to the output u_theta(x,t). A single neuron computes z = w1 x1 + w2 x2 + b, followed by an activation function a = sigma(z). Stacking layers, the first hidden layer is z1 = W1 x + b1 with activation h1 = sigma(z1), repeating through subsequent layers until the final output u_theta(x,t) = W(L) h(L-1) + b(L).

3.2 Automatic Differentiation and the Physics Residual

Because the PINN must also satisfy the governing Burgers' equation, its output is differentiated with respect to its inputs using automatic differentiation to obtain u_t = du_theta/dt, u_x = du_theta/dx, and u_xx = d2u_theta/dx2. Substituting these into Burgers' equation defines the physics residual:

f_theta(x,t) = du_theta/dt + u_theta du_theta/dx - nu d2u_theta/dx2

If f_theta(x,t) = 0, the network's prediction satisfies the governing equation perfectly; if f_theta(x,t) is not equal to 0, the network violates the physics. The training objective is to drive f_theta(x,t) toward zero at a set of collocation points sampled within the computational domain.

3.3 Loss Function

The total loss function combines three terms: L = L_PDE + L_IC + L_BC.

- Physics equation loss: L_PDE is the mean squared residual over Nf collocation points, L_PDE = (1/Nf) * sum |f_theta(x_f^i, t_f^i)|^2. Small values indicate the predicted solution follows the physics more closely.
- Initial condition loss: L_IC = (1/N0) * sum |u_theta(x0^i, 0) - u0(x0^i)|^2, forcing the network to start from the correct velocity distribution rather than an arbitrary initial state.
- Boundary condition loss: L_BC = (1/Nb) * sum |u_theta(xb^i, tb^i) - g(xb^i, tb^i)|^2, enforcing the prescribed boundary values.

The network parameters are updated by minimizing the combined loss: theta = argmin_theta L.

4. Quantum-Assisted Physics-Informed Neural Network (QAPINN)

QAPINNs extend the classical PINN framework by integrating a Variational Quantum Circuit (VQC) into the learning process. The governing physical laws, initial conditions, and boundary conditions remain embedded in

the loss function exactly as in the classical PINN; the quantum circuit instead provides additional feature extraction and learning capability by replacing one or more classical layers.

4.1 Model Architecture

Both the Classical PINN (cPINN) and the Quantum-Assisted PINN (QAPINN) shared a foundational architecture to ensure a fair comparison. The models take a 2D input space (x, t) and output a scalar fluid velocity $u(x, t)$. The hidden layers utilized the Hyperbolic Tangent (Tanh) activation function, which provides the necessary continuous second-order derivatives required by the PyTorch autograd engine to compute the Burgers' equation residual. In the QAPINN, the first classical hidden layer was replaced by a PennyLane Variational Quantum Circuit (VQC) utilizing an Angle Embedding strategy, interfaced via qml.qnn.TorchLayer.

4.2 Variational Quantum Circuit (VQC)

A VQC is the quantum equivalent of a neural network: rather than adjusting weights and biases, it learns by adjusting the parameters of quantum gates. Its role is to transform classical input data into a quantum state, perform trainable quantum operations, and produce an output usable by subsequent classical layers.

Encoding Classical Inputs

Since a quantum computer cannot directly process ordinary numbers, the classical inputs (x,t) are first encoded into a quantum state |psi(x,t)> = U(x,t)|0> using angle encoding via rotation gates Ry(x) and Ry(t), each rotating a qubit on the Bloch sphere by an angle equal to the corresponding input value.

Trainable Quantum Gates and Entanglement

After encoding, trainable quantum gates (e.g. Rx(theta1), Ry(theta2), Rz(theta3)) are applied to each qubit, with parameters phi optimized during training via a trainable circuit V(phi). CNOT gates entangle the qubits, allowing the circuit to represent more complex relationships between inputs than independent single-qubit rotations could capture.

Measurement

The expectation value of the Pauli-Z observable is measured for each qubit, converting the quantum state back into a classical value q(x,t) that can be passed to subsequent classical layers.

4.3 Training Hyperparameters and Optimization

The networks were trained using the Adam optimizer (torch.optim.Adam) with a fixed learning rate of 0.01. The training loop was executed for 1000 epochs. The loss function was formulated as the unweighted sum of the Mean Squared Error (MSE) of the initial conditions, boundary conditions, and the PDE residual.

4.4 Collocation Points and Data

SamplingBecause PINNs are trained via physics-informed residuals rather than external datasets, collocation points were dynamically sampled using a uniform random distribution at each epoch. The spatial domain was defined as $x \in [-1, 1]$ and the temporal domain as $t \in [0, 1]$.For each forward pass, the following points were sampled:PDE Residual (N_f): 100 internal collocation points.Initial Condition (N_{IC}): 50 points evaluated at $t = 0$.Boundary Conditions (N_{BC}): 100 points total (50 on the left boundary $x = -1$, and 50 on the right boundary $x = 1$).For the final XAI evaluation, the trained models were inferenced on a static 256×100 uniform grid to perfectly match the dimensions of the exact analytical solution provided by Raissi et al., ensuring zero interpolation error during L2 metric calculation.

4.5 Hardware

All experiments, including quantum circuit simulations, were executed using Google Colab utilizing an NVIDIA T4 Tensor Core GPU.

4.6 Hybrid Quantum-Classical Architecture

The measured quantum feature q(x,t) is combined with classical parameters theta in subsequent classical layers to produce the final prediction u_(theta,phi)(x,t). This hybrid design keeps the classical PINN loss formulation intact while introducing quantum-derived features into the network's representation.

4.7 Training Algorithm

Training jointly optimizes the classical parameters (theta) and quantum circuit parameters (phi) against the same physics-informed loss function used for the classical PINN, following these steps:

- Define the computational domain and sample collocation, initial, and boundary points.
- Encode (x,t) into a quantum state via angle embedding.
- Process the encoded state through the trainable VQC.
- Measure Pauli-Z expectation values and pass the result to the classical network.
- Predict u_(theta,phi)(x,t) via the classical output layer.
- Compute the required derivatives via automatic differentiation.
- Evaluate the physics residual and the combined loss L = L_PDE + L_IC + L_BC.
- Update both classical and quantum parameters via gradient-based optimization.
- Repeat until convergence.

5. Experimental Protocol

Both the classical PINN and the QAPINN configurations (3, 4, and 5 qubits) were trained in Google Colab using a T4 GPU, with PyTorch automatic differentiation and PennyLane for the quantum circuit simulation. To ensure reproducibility, a fixed random seed (seed=42) was enforced across PyTorch, NumPy, and Python random initializations. Each model was trained for 1000 epochs.

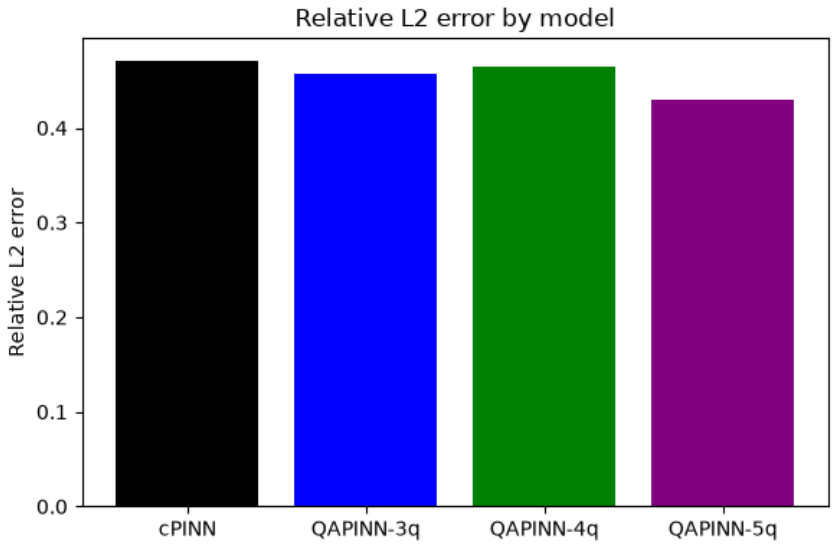
Specific optimizer choice, learning rate, and the exact number of collocation/initial/boundary points used during training were not available in the documentation reviewed for this report; these details should be added by the implementation lead if a full reproducibility record is required beyond the published code repository.

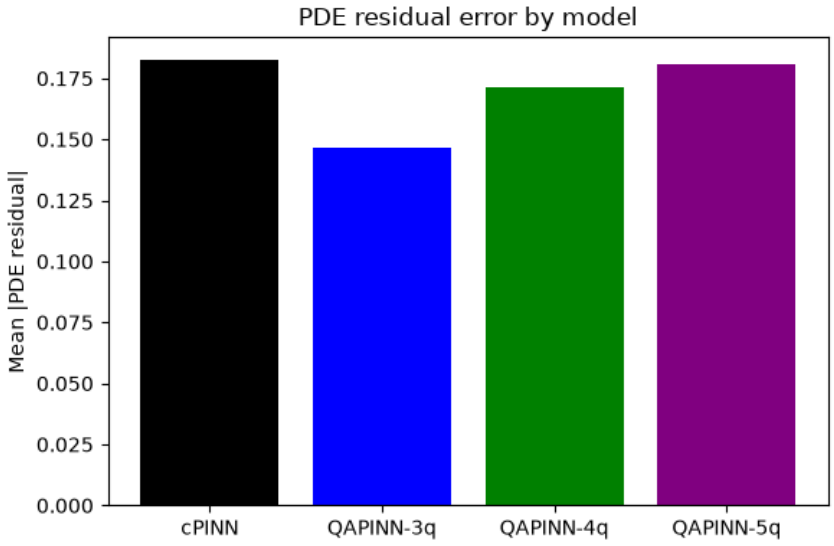
6. Results

6.1 Quantitative Comparison

Metric	cPINN	QAPINN-3q	QAPINN-4q	QAPINN-5q
Relative L2 error	0.4711	0.4571	0.4640	0.4293
Mean |PDE residual|	0.1827	0.1469	0.1716	0.1809
Trainable parameters	573	548	577	606
Activation diversity (layer 2)	184.5	207.3	206.5	196.1
Activation diversity (layer 3)	204.2	209.5	215.8	207.4

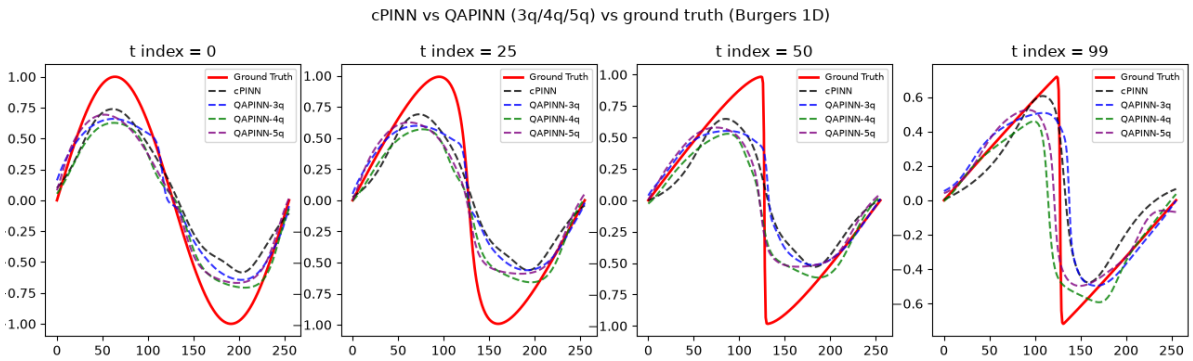
All three QAPINN configurations outperform the classical baseline on relative L2 error, with the 5-qubit variant achieving the lowest error (0.4293 vs. 0.4711, a ~9% relative improvement). No single QAPINN configuration dominates every metric: the 3-qubit variant achieves the best PDE residual and is the only configuration with fewer trainable parameters than the cPINN baseline (548 vs. 573). Activation diversity is higher for every QAPINN configuration than for the cPINN in both hidden layers -- the most consistent signal across the experiment.





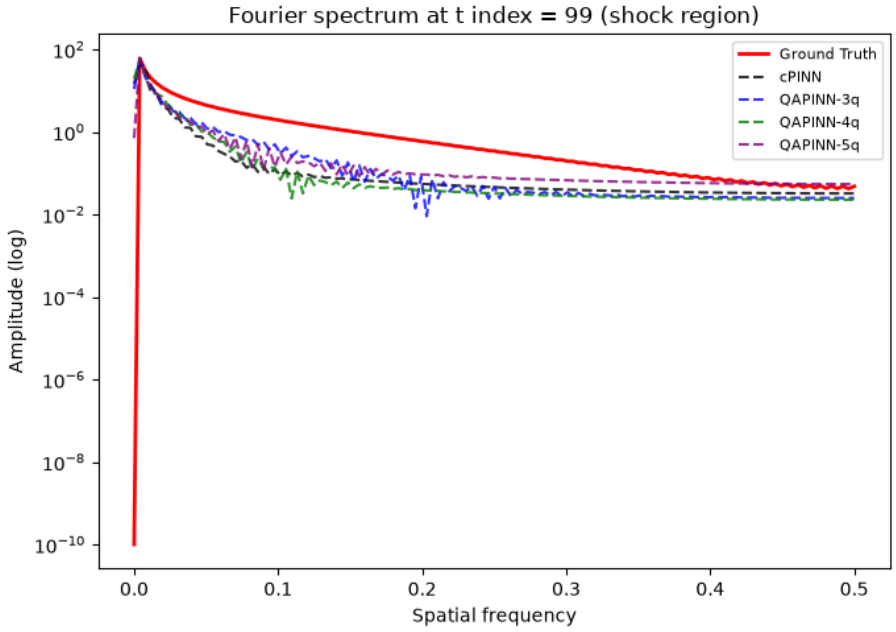
6.2 Prediction Accuracy Over Time

All four models track the ground truth closely at early time steps but fail to capture the sharp shock that forms later in the simulation (from t index = 50 onward), remaining smoother than the true solution near the discontinuity. This limitation, consistent with the nonlinear convection term's tendency to produce steep gradients described in Section 2.1, is shared across the classical and quantum-assisted models alike.



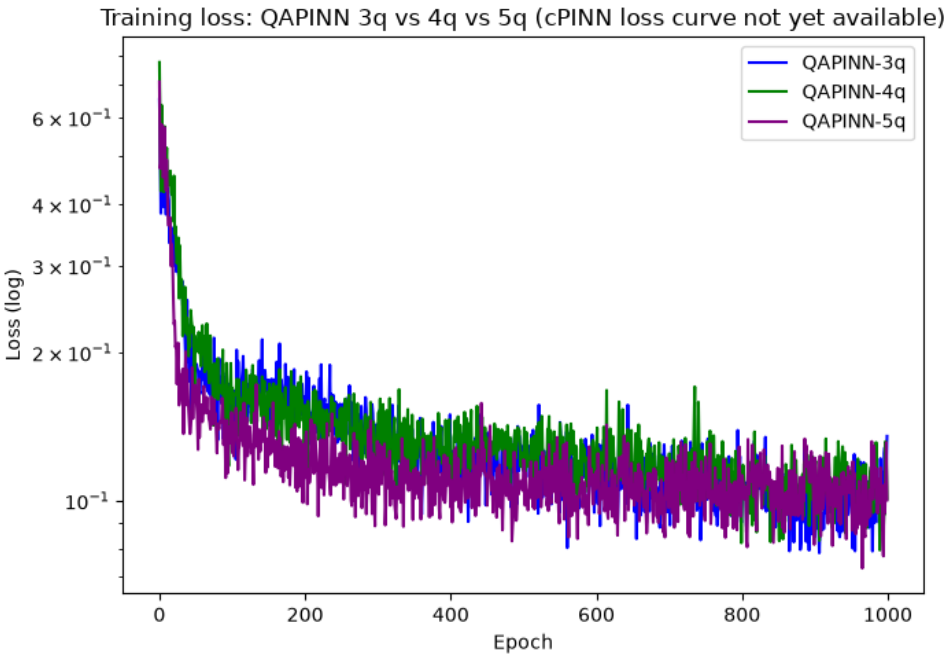
6.3 Fourier Spectral Analysis

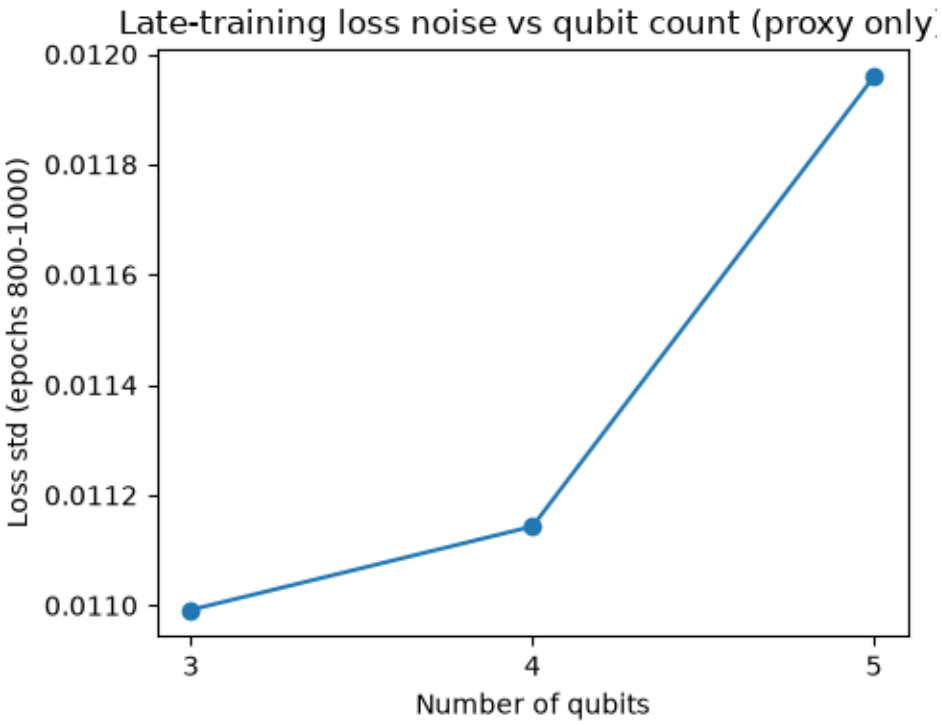
At the final time step (shock fully formed), the ground truth's high-frequency spectral energy ratio (0.0039) is one to two orders of magnitude higher than for any trained model (cPINN: 0.0001; QAPINN-3q: 0.0001; QAPINN-4q: 0.0000; QAPINN-5q: 0.0001). This confirms quantitatively that none of the models -- regardless of the presence or size of a quantum layer -- reproduce the shock's high-frequency content. This is consistent with the choice of angle encoding described in Section 4.2, which, per Schuld and Killoran's feature-Hilbert-space interpretation of quantum encodings [6], determines the frequency spectrum accessible to the model; a richer or higher-frequency encoding scheme may be a promising direction for improving shock resolution in future work.



6.4 Training Dynamics and Loss Noise

QAPINN loss curves (3q/4q/5q) converge to a similar final loss (~0.1, log scale) after 1000 epochs. Late-training loss noise (standard deviation over epochs 800-1000) increases mildly with qubit count: 3q = 0.0110, 4q = 0.0111, 5q = 0.0120. This is reported as an indirect proxy only, not a true barren plateau test, which would require direct VQC parameter gradient sampling not available from the current data exports.





7. Discussion

- The quantum layer provides a modest but consistent accuracy improvement across all tested qubit counts, without requiring more parameters in the 3-qubit case.
- Increasing qubit count does not uniformly improve all metrics -- different configurations are preferable depending on whether L2 accuracy, PDE residual, or parameter efficiency is prioritized.
- The quantum layer's most consistent effect is an increase in downstream activation diversity in the classical layers that follow it, plausibly linked to the entangling CNOT structure described in Section 4.2 producing richer joint feature representations than an equivalent classical layer.

- Both architectures share the same fundamental limitation on the sharp shock region of Burgers' equation -- a property of the PINN loss formulation itself (Section 3.3), not something the quantum layer resolves or worsens.

8. Limitations and Future Work

- Fourier spectral analysis and loss dynamics are complete; a rigorous barren plateau / gradient variance analysis is not, due to data export constraints (Section 6.4).
- Optimizer, learning rate, and collocation point counts were not available for this report and should be documented for full reproducibility (Section 5).
- Relative L2 error values differ slightly between two independent training runs of the same models; this run-to-run variance should be characterized (e.g. averaged over multiple seeds) if time permits.
- Results are specific to the 1D viscous Burgers' equation; generalization to other PDE classes (heat equation, other CFD problems) is untested.

9. Conclusion

This study presented the full mathematical derivation of the one-dimensional viscous Burgers' equation from the Navier-Stokes equation, formulated both a classical PINN and a Quantum-Assisted PINN (QAPINN) using a variational quantum circuit with angle encoding and entangling gates, and compared their performance using Explainable AI techniques. Across 3, 4, and 5-qubit configurations, the QAPINN consistently achieved lower relative L2 error and higher activation diversity than the classical baseline, without a uniform improvement across every metric. Both architectures share the same core limitation: neither resolves the sharp shock characteristic of Burgers' equation, a limitation rooted in the PINN loss formulation rather than the presence of a quantum layer. These findings support a nuanced view of quantum-assisted PINNs -- offering measurable but modest benefits under the tested configuration, with clear directions (encoding scheme, gradient-based plateau analysis, multi-PDE generalization) for further investigation.

References

[1] Hu, Jagtap, Karniadakis, Kawaguchi. When do extended physics-informed neural networks (xpinns) improve generalization? SIAM J. Sci. Comput., 44(5):A3158-A3182, 2022.

[2] Raissi, Perdikaris, Karniadakis. Physics informed deep learning (part I): Data-driven solutions of nonlinear partial differential equations. arXiv:1711.10561, 2017.

[3] Shah, Lineswala, Chopra. Benchmarking quantum-assisted pinn (qa-pinn) for computational fluid dynamics. IEEE QCE 2024.

[4] Schuld, Sweke, Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. Phys. Rev. A 103, 032430, 2021.

[5] Bergholm, V. et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. arXiv:1811.04968, 2018.

[6] Schuld, M. and Killoran, N. Quantum machine learning in feature Hilbert spaces. Phys. Rev. Lett. 122, 040504, 2019.

Appendix A: Reproducibility

Appendix B: Internal Layer Activation Maps
Code repository: github.com/mas622424/WISER-BQP-QAPINN. Environment: pip install torch pennylane numpy matplotlib scipy. Reproducibility: seed=42 enforced across PyTorch, NumPy, and Python random initializations. Training: Google Colab, T4 GPU, Runtime > Run all.

Appendix B: Internal Layer Activation Maps

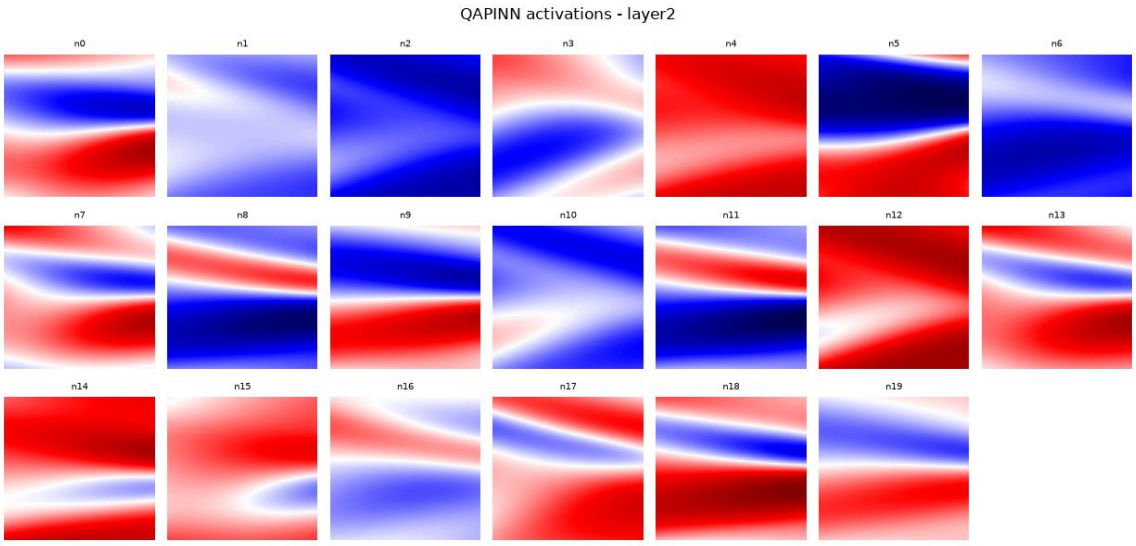


Figure A.1: Spatial-temporal activation maps for hidden neurons in QAPINN layer 2.

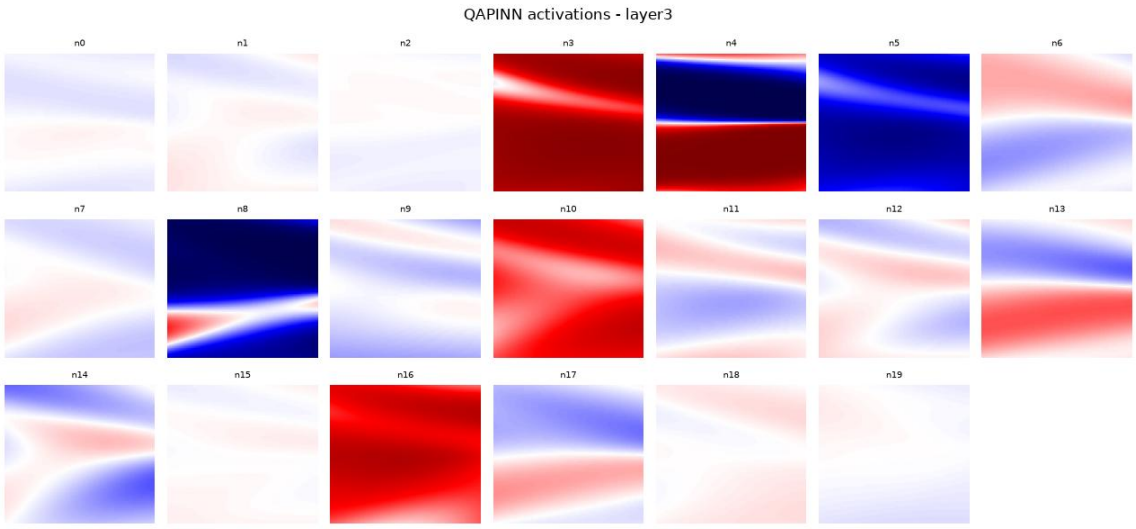


Figure A.2: Spatial-temporal activation maps for hidden neurons in QAPINN layer 3.